Vero

Documento de Arquitetura

Versão 1.0

Tabela - Integrantes do Grupo

Matrícula	Nome	Função (responsabilidade)	Pontos de participação na elaboração
251018246	Andris G.	PO	16
242005202	Beatriz B.	Analista de Qualidade, Arquitetura	18
232002735	Gabriel M.	Desenvolvedor Backend, Cliente	13
231012209	Gabriel S.	Desenvolvedor Backend, Cliente	13
231012254	João V.	Desenvolvedor Backend	13
242005436	Ryan	Desenvolvedor Frontend, Scrum Master	16
24103979	Sabrina	Desenvolvedora Frontend	12

Histórico de Revisões

Data	Versão	Descrição	Autor(es)
04/10/2026	1.0	Versão inicial: estilo arquitetural, pilha open source, visões, diagramas e esteira de CI/CD	Beatriz Brandão

Sumário

1 Introdução 4
1.1 Propósito 4
1.2 Escopo 4
2 Representação Arquitetural 4
2.1 Definições 4
2.2 Justificativa da escolha 5
2.3 Detalhamento 7
2.4 Metas e restrições arquiteturais 9
2.5 Visões 10
2.5.1 Visão de uso 10
2.5.2 Visão de organização lógica 13
2.5.3 Visão estrutural 15
2.6 Visão de Implantação 16
2.7 Restrições adicionais 18
3 Bibliografia 20
Apêndice A – Padrões de codificação e de repositório 21

1 Introdução

1.1 Propósito

Este documento descreve a arquitetura do **Vero**, sistema desenvolvido pelo grupo **Allen** na disciplina de MDS – Métodos de Desenvolvimento de Software, turma T03, edição do segundo semestre de 2026 (2026/2). Seu objetivo é fornecer uma visão abrangente do sistema para desenvolvedores, testadores e demais interessados nas tecnologias usadas no desenvolvimento, registrando cada decisão arquitetural, sua justificativa e as visões de uso, lógica, estrutural e de implantação.

Por decisão do grupo, **todo o software que compõe e sustenta o Vero é de código aberto**, distribuído sob licenças aprovadas pela Open Source Initiative (OSI). As únicas exceções são serviços externos que o próprio problema ou a disciplina impõem — a rede do WhatsApp, o portal de consulta da nota fiscal (SEFAZ) e o GitHub — e elas estão justificadas na seção 2.7.

1.2 Escopo

O detalhamento do escopo encontra-se no **Documento de Visão do Produto e do Projeto** (entregue em 30/09/2026), juntamente com os documentos **Requisitos de Software v2**, **Backlog v2** e **Planejamento v2.** Em linhas gerais, o Vero é um sistema de organização financeira pessoal **aberto a qualquer pessoa**, projetado com **foco em pessoas idosas** — e, por extensão, em pessoas com baixo letramento, dificuldade visual ou pouca familiaridade com tecnologia. As escolhas feitas para esse público prioritário (interface acessível, linguagem simples, entrada por voz e por foto) tornam o sistema mais fácil para todos os usuários. O Vero é formado por dois canais que compartilham os mesmos dados:

- **Aplicativo web (PWA)** responsivo e acessível, no qual o usuário vê o dinheiro disponível no mês em números grandes e em uma barra colorida, registra gastos com um teclado numérico grande ou pela leitura do QR Code da nota fiscal, e acompanha as contas a pagar em lista ou calendário.
- **Assistente no WhatsApp**, que registra gastos, receitas e contas a partir de mensagens de texto, áudio ou foto — sempre confirmando com o usuário antes de salvar —, responde perguntas sobre o saldo e envia avisos de vencimento e resumos periódicos.

O escopo compreende 34 requisitos funcionais (RF01–RF34) e 5 requisitos não funcionais (RNF01–RNF05), distribuídos em 8 sprints semanais, de 07/10/2026 a 29/11/2026, mais uma Sprint 0 (30/09 a 07/10) dedicada à arquitetura e à esteira de integração e entrega contínuas (CI/CD).

2 Representação Arquitetural

2.1 Definições

O sistema seguirá uma **arquitetura cliente-servidor em camadas**, com o back-end organizado como um **monólito modular** e as integrações externas isoladas pelo padrão **Portas e Adaptadores** (arquitetura hexagonal). Tarefas demoradas — leitura de imagens e transcrição de áudios — e rotinas agendadas são executadas por **processamento assíncrono baseado em fila**. Cada um desses elementos tem um papel específico:

- **Cliente-servidor em camadas:** o sistema é dividido em camada de apresentação (PWA e conversa no WhatsApp), camada de aplicação (API) e camada de dados. Cada camada só se comunica com a camada vizinha.
- **Monólito modular:** existe uma única API, implantada como uma única unidade, mas internamente dividida em módulos de domínio (Identidade, Transações, Contas a Pagar, Painel, Assistente e Notificações) com fronteiras explícitas.
- **Portas e Adaptadores:** o núcleo da aplicação define interfaces (portas) para tudo o que é externo — mensageria do WhatsApp, leitura de imagem (OCR), transcrição de áudio e consulta de nota fiscal. Adaptadores trocáveis implementam essas portas.
- **Processamento assíncrono:** a API coloca tarefas pesadas em uma fila; processos separados (workers) as executam e devolvem o resultado ao módulo Assistente. Um agendador dispara as rotinas diárias (avisos, resumos e lançamentos recorrentes).

2.2 Justificativa da escolha

A escolha foi guiada pelos requisitos registrados nos documentos de Requisitos de Software v2 e Backlog v2, pela restrição de usar somente ferramentas de código aberto e pelo contexto do projeto (equipe de estudantes, sprints semanais e prazo de um semestre). O Quadro 1 relaciona cada fator determinante à decisão que ele motivou.

Quadro 1 – Fatores determinantes e decisões arquiteturais

Fator determinante	Decisão	Justificativa
Dois canais de entrada (app e WhatsApp) atualizando os mesmos dados, refletidos na hora (RF17)	Cliente-servidor com uma única API como fonte da verdade	Os dois canais gravam e leem no mesmo servidor; não há sincronização entre cópias de dados. A atualização em tempo real é feita por eventos enviados ao app via WebSocket.
Equipe pequena, 8 sprints semanais e um único ambiente por estágio (staging e produção)	Monólito modular em vez de microsserviços	Um só deploy, uma só base de código e transações locais no banco reduzem o custo de operação. As fronteiras entre módulos permitem extrair um módulo no futuro, se necessário (FOWLER, 2015).
Integrações sujeitas a mudança: WhatsApp, motores de OCR e de transcrição, portal da SEFAZ	Portas e Adaptadores	Trocar o provedor (por exemplo, outro gateway de WhatsApp) não altera as regras de negócio. No CI, os adaptadores são substituídos por versões falsas, e o assistente é testado sem enviar mensagens reais (COCKBURN, 2005).
Leitura de foto e áudio em até 5 segundos (RNF04)	Workers assíncronos com modelos pré-carregados	O processamento pesado não bloqueia a API, os modelos ficam carregados em memória entre as tarefas e o número de workers pode ser aumentado sem mudar o código.
Acessibilidade WCAG 2.1 AA, linguagem simples e responsividade (RNF01–RNF03)	Aplicativo web progressivo (PWA)	Uma só base de código atende Android e iOS, sem instalação por loja. Recursos padrão do navegador cobrem câmera (QR Code), leitura em voz (Web Speech API) e biometria (WebAuthn).
Somente ferramentas de código aberto	Pilha 100% OSI (Quadro 2)	Todas as bibliotecas, servidores e ferramentas de teste têm licença aprovada pela OSI e podem ser auditadas e executadas no próprio servidor do grupo.

Fonte: elaborado pelo grupo (2026).

O Quadro 2 apresenta a pilha tecnológica, a função de cada ferramenta no Vero e a respectiva licença.

Quadro 2 – Pilha tecnológica open source e licenças

Camada	Ferramenta	Função no Vero	Licença
Apresentação	React + TypeScript	Interface do PWA	MIT / Apache-2.0
Apresentação	Vite + vite-plugin-pwa (Workbox)	Build, instalação na tela inicial e cache offline	MIT
Apresentação	ZXing (@zxing/library)	Leitura do QR Code da NFC-e (RF07)	Apache-2.0
Apresentação	SimpleWebAuthn	Login por biometria via WebAuthn (RF02)	MIT
Borda	Caddy	HTTPS automático, proxy reverso e entrega do PWA	Apache-2.0
Aplicação	Python + FastAPI + Uvicorn	API REST, WebSocket e webhook do WhatsApp	PSF-2.0 / MIT / BSD-3-Clause
Aplicação	SQLAlchemy + Alembic	Acesso ao banco e migrações versionadas	MIT
Aplicação	Pydantic	Validação dos dados de entrada	MIT
Aplicação	argon2-cffi, PyJWT, py_webauthn	Hash do PIN, tokens de sessão e verificação WebAuthn	MIT / MIT / BSD-3-Clause
Assíncrono	Celery + Celery Beat	Fila de tarefas e agendamentos diários	BSD-3-Clause
IA	faster-whisper (modelo Whisper)	Transcrição de áudios (RF13)	MIT
IA	PaddleOCR	Leitura de comprovantes e contas (RF14, RF27)	Apache-2.0
IA	RapidFuzz	Associação de palavras às categorias (RF12)	MIT
Integração	httpx + Beautiful Soup	Consulta e leitura da NFC-e na SEFAZ (RF07)	BSD-3-Clause / MIT
Dados	PostgreSQL	Banco de dados relacional	PostgreSQL License
Dados	Valkey	Fila de tarefas, eventos em tempo real e cache	BSD-3-Clause
Infraestrutura	Debian, Docker Engine, Docker Compose	Sistema operacional e contêineres	Livre (DFSG) / Apache-2.0
Infraestrutura	LUKS (cryptsetup)	Criptografia do disco do servidor	GPL-2.0
Qualidade	pytest, coverage.py, Vitest, Testing Library	Testes automatizados e cobertura	MIT / Apache-2.0 / MIT / MIT
Qualidade	Playwright, axe-core, Lighthouse CI	Testes ponta a ponta e de acessibilidade	Apache-2.0 / MPL-2.0 / Apache-2.0
Qualidade	Ruff, ESLint, Prettier	Lint e formatação	MIT
Segurança	Bandit, Gitleaks, Trivy	Análise estática, segredos e vulnerabilidades	Apache-2.0 / MIT / Apache-2.0
Manutenção	Renovate	Atualização automática de dependências	AGPL-3.0
Documentação	Mermaid, draw.io	Diagramas versionados no repositório	MIT / Apache-2.0

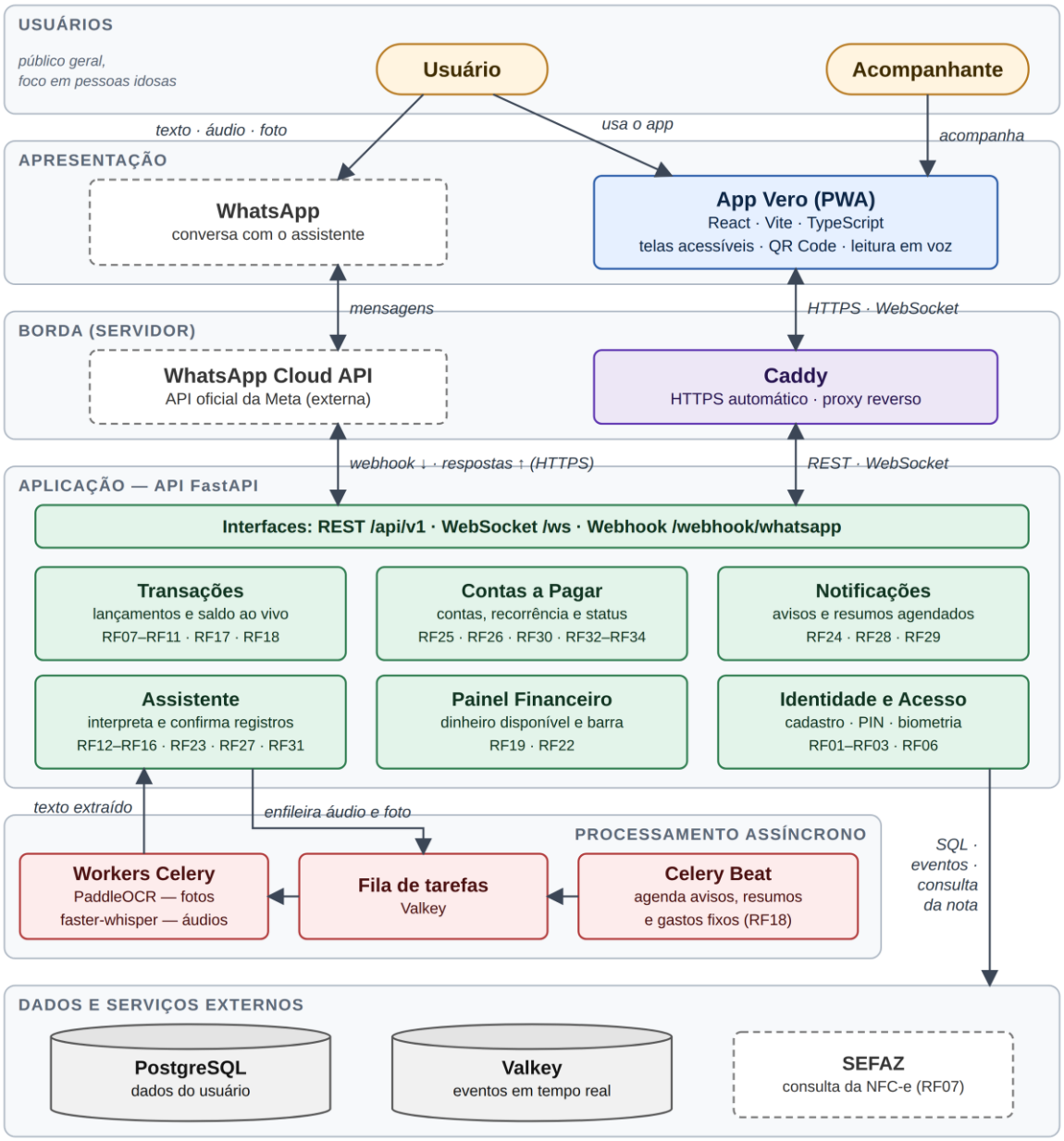
Fonte: elaborado pelo grupo (2026).

O gateway Evolution API, comum em projetos brasileiros com WhatsApp, foi avaliado e **descartado**: sua licença acrescenta condições ao Apache-2.0 (manutenção de marca e aviso de uso), o que a afasta da definição de código aberto adotada pelo grupo.

2.3 Detalhamento

A Figura 1 é o fluxograma da arquitetura. Ela deve ser lida de cima para baixo: os usuários interagem por dois canais; as requisições entram no servidor pela borda; a API distribui o trabalho entre os módulos; o que é demorado vai para a fila; e os dados ficam na camada inferior.

Figura 1 – Fluxograma da arquitetura do Vero



Fonte: elaborado pelo grupo (2026).

O Quadro 3 instancia os elementos do estilo arquitetural para o Vero e explica a responsabilidade de cada um, as interfaces que oferece e as regras de uso.

Quadro 3 – Elementos da arquitetura, responsabilidades e interfaces

Elemento	Responsabilidade	Interfaces e regras de uso
App Vero (PWA)	Telas acessíveis, teclado numérico, leitura do QR Code, leitura em voz e confirmação visual (RF04, RF05, RF07–RF11, RF19–RF22, RF25–RF34).	Consome a API REST em /api/v1 e recebe eventos pelo WebSocket /ws. Não guarda dados financeiros no aparelho além do cache da sessão.
WhatsApp Cloud API (externo)	Entrega as mensagens do usuário ao Vero e as respostas do assistente ao usuário.	Chama o webhook HTTPS /webhook/whatsapp, assinado pela Meta. Mensagens proativas (avisos e resumos) exigem modelos aprovados.
Caddy	Termina o HTTPS (certificados automáticos), entrega os arquivos do PWA e encaminha /api, /ws e /webhook para a API.	Única porta aberta para a internet (443). Aplica cabeçalhos de segurança e limite de tamanho de requisição.
API FastAPI	Ponto único de entrada da aplicação: autenticação, validação, roteamento para os módulos e publicação de eventos.	REST/JSON versionado (/api/v1) e documentado em OpenAPI; WebSocket autenticado por token; webhook com verificação de assinatura.
Módulo Identidade e Acesso	Cadastro por celular, PIN, biometria, sessões e vínculo com acompanhante (RF01–RF03, RF06).	Entrada: telefone, código de verificação, PIN ou credencial WebAuthn. Saída: token de sessão com o perfil (titular ou acompanhante).
Módulo Transações	Registrar, editar, excluir e desfazer lançamentos; consultar NFC-e; lançar gastos fixos; publicar o evento "transação criada" (RF07–RF11, RF17, RF18).	Único módulo que grava transações. Os demais módulos o chamam pela sua interface de serviço, nunca diretamente pelas tabelas.
Módulo Contas a Pagar	Cadastro de contas e contas fixas, status (a vencer, paga, em atraso), lista diária, filtros e calendário (RF25, RF26, RF30, RF32–RF34).	Status calculado no fuso de Brasília. Expõe consulta de contas por período, usada pelo calendário e pelas notificações.
Módulo Painel Financeiro	Calcula o dinheiro disponível no mês e o percentual já gasto (RF19, RF22).	Somente leitura. É o mesmo cálculo usado pelo app e pelo assistente (RF23), o que evita números diferentes nos dois canais.
Módulo Assistente	Interpreta a intenção, extrai valor, tipo e categoria, conduz a confirmação e a correção (RF12–RF16, RF23, RF27, RF31).	Máquina de estados por conversa (SessaoAssistente). Só grava depois do "Sim" do usuário. Usa as portas de Mensageria, OCR e Transcrição.
Módulo Notificações	Avisos de contas a vencer e resumos periódicos (RF24, RF28, RF29).	Disparado pelo agendador; envia pelo app e pela Porta de Mensageria. Não envia aviso para conta já paga nem aviso duplicado.
Fila + Workers (Celery)	Executar OCR (PaddleOCR) e transcrição (faster-whisper) fora da requisição.	Tarefas idempotentes, com tempo limite e novas tentativas. O resultado volta ao Assistente.
Celery Beat	Agendar as rotinas diárias: avisos, resumos e lançamentos recorrentes.	Horários definidos no fuso America/Sao_Paulo.
PostgreSQL	Persistir usuários, transações, contas, regras de recorrência e sessões do assistente.	Acessado somente pela API e pelos workers, pela rede interna. Valores monetários guardados em centavos inteiros.

Elemento	Responsabilidade	Interfaces e regras de uso
Valkey	Fila de tarefas, canal de eventos em tempo real e cache.	Não guarda dados de forma permanente; pode ser reiniciado sem perda de informação financeira.
SEFAZ (externo)	Fornece os dados da NFC-e lida pelo QR Code.	Consulta pública por HTTPS; falhas tratadas com mensagem simples e opção de digitar o valor.

Fonte: elaborado pelo grupo (2026).

2.4 Metas e restrições arquiteturais

O Quadro 4 lista as metas de funcionamento, como a arquitetura as atende e como cada uma é verificada. Todas são checadas automaticamente na esteira de CI/CD ou nos testes de aceite feitos pela QA a cada sprint.

Quadro 4 – Metas arquiteturais

Meta	Como a arquitetura atende	Como é verificada
M1 – Acessibilidade WCAG 2.1 AA (RNF01)	Biblioteca de componentes acessíveis: contraste mínimo de 4,5:1, fonte base de 18 px, alvos de toque de pelo menos 44 px, foco visível, rótulos para leitor de tela e informação nunca transmitida só por cor.	axe-core em todo PR (falha em violação séria ou crítica); Lighthouse CI com nota mínima de 95 em acessibilidade; roteiro manual com leitor de tela e zoom de 200%.
M2 – Linguagem simples (RNF02)	Todos os textos do app e do assistente ficam em arquivos de mensagens, fora do código, seguindo o guia de linguagem do projeto.	Revisão dos arquivos de mensagens pela QA em cada PR.
M3 – Responsividade (RNF03)	Layout mobile-first, testado de 360 px a 1280 px de largura.	Testes ponta a ponta (Playwright) executados em viewport de celular.
M4 – Resposta do assistente em até 5 s (RNF04)	Workers com modelos carregados na inicialização e quantizados (int8); aviso imediato de "recebi, estou lendo"; áudios limitados a 60 s; imagens reduzidas antes do OCR.	Medição do percentil 95 com pelo menos 30 amostras de áudio e 30 de foto, a partir da Sprint 4 e de novo na Sprint 8.
M5 – Atualização em tempo real (RF17)	Evento "transação criada" publicado no Valkey e repassado ao app pelo WebSocket.	Teste de integração: registro pelo webhook aparece na tela em até 2 s.
M6 – Segurança e LGPD (RNF05)	Criptografia em trânsito e em repouso e minimização de dados (detalhado na seção 2.7).	Bandit, Gitleaks e Trivy em todo PR; auditoria final na Sprint 8.
M7 – Integridade da branch principal	Nenhum código entra na main sem PR revisado e CI verde.	Proteção de branch no GitHub; cobertura mínima de 70% no back-end.
M8 – Somente software livre	Toda nova dependência precisa ter licença aprovada pela OSI.	Revisão no PR e varredura de licenças com o Trivy.

Fonte: elaborado pelo grupo (2026).

Padrão de API. A API segue o estilo REST com JSON, versionado no caminho (/api/v1) e documentado automaticamente em OpenAPI pelo FastAPI. Os erros seguem o formato Problem Details da RFC 9457 (NOTTINGHAM; WILDE; DALAL, 2023), com uma mensagem em linguagem simples que o app mostra diretamente ao usuário. Datas usam ISO

8601, o fuso de referência é America/Sao_Paulo e valores monetários trafegam em centavos inteiros, o que elimina erros de arredondamento.

Padrão de codificação. O grupo adota os padrões descritos no Apêndice A, verificados automaticamente pela esteira de CI.

2.5 Visões

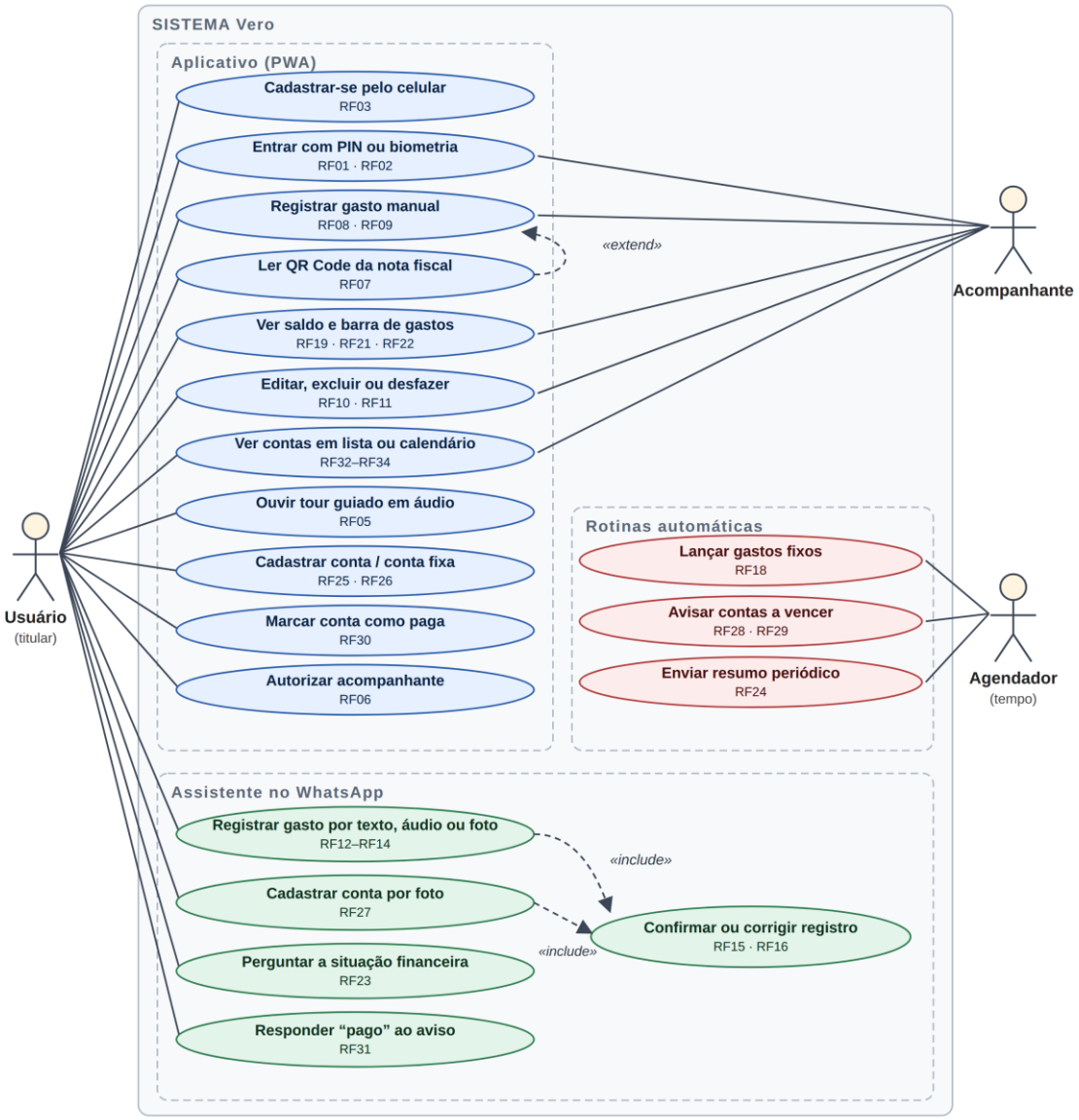
A arquitetura é comunicada por quatro visões complementares, inspiradas no modelo 4+1 de Kruchten (1995):

- **Visão de uso:** o escopo do sistema, os atores e o fluxo mais complexo (seção 2.5.1).
- **Visão de organização lógica:** os módulos e as camadas da arquitetura (seção 2.5.2).
- **Visão estrutural:** a composição do sistema e as relações entre suas partes (seção 2.5.3).
- **Visão de implantação:** o ambiente em que o sistema é executado (seção 2.6).

2.5.1 Visão de uso

Em poucas palavras, o Vero permite que qualquer pessoa — em especial a pessoa idosa, público prioritário do projeto — saiba quanto dinheiro ainda tem no mês e não esqueça suas contas, usando o meio que lhe for mais confortável: um aplicativo simples ou uma conversa no WhatsApp. A Figura 2 apresenta o diagrama de casos de uso com os três atores do sistema.

Figura 2 – Diagrama de casos de uso do Vero

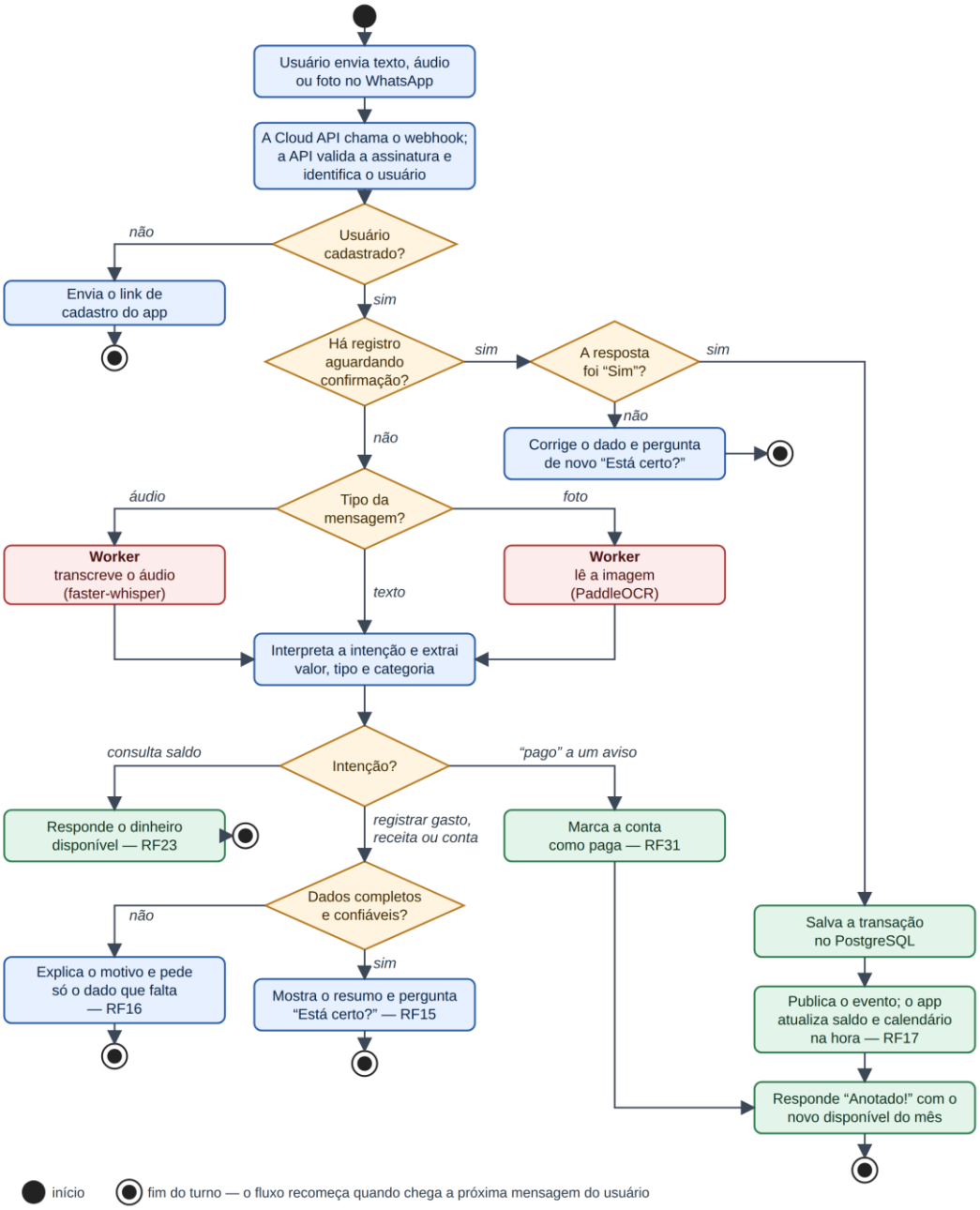


Fonte: elaborado pelo grupo (2026).

- **Usuário (titular):** dono da conta; usa o aplicativo e o assistente. Pode ser qualquer pessoa; o público prioritário são as pessoas idosas, e por isso a interface segue os critérios de acessibilidade e de linguagem simples descritos na seção 2.4.
- **Acompanhante:** pessoa de confiança autorizada pelo titular (RF06) — por exemplo, um familiar que ajuda um usuário idoso; pode ver o saldo e as contas e registrar ou corrigir lançamentos, mas não altera o acesso do titular.
- **Agendador (tempo):** ator que representa as rotinas automáticas disparadas por horário.

A funcionalidade mais complexa é o **registro de um lançamento pelo assistente no WhatsApp**: ela envolve três formatos de entrada (texto, áudio e foto), processamento assíncrono, a confirmação obrigatória antes de salvar (RF15), o tratamento de falhas de leitura (RF16) e a atualização imediata do aplicativo (RF17). A Figura 3 mostra o diagrama de atividades desse fluxo.

Figura 3 – Diagrama de atividades do registro pelo assistente no WhatsApp



Fonte: elaborado pelo grupo (2026).

Alguns requisitos foram determinantes para o estilo arquitetural escolhido:

- **RF17** exige uma fonte única de dados para os dois canais, o que leva a uma única API.
- **RF12 a RF16 e RNF04** exigem processamento pesado com resposta rápida, o que leva aos workers assíncronos.
- **RF13, RF14 e RF27** dependem de motores de IA que podem ser trocados, o que leva ao padrão Portas e Adaptadores.
- **RNF01 a RNF03** exigem uma interface acessível em qualquer celular, o que leva ao PWA.

2.5.2 Visão de organização lógica

O sistema é subdividido nos seguintes módulos, organizados em pacotes conforme a Figura 4:

Quadro 5 – Módulos do sistema

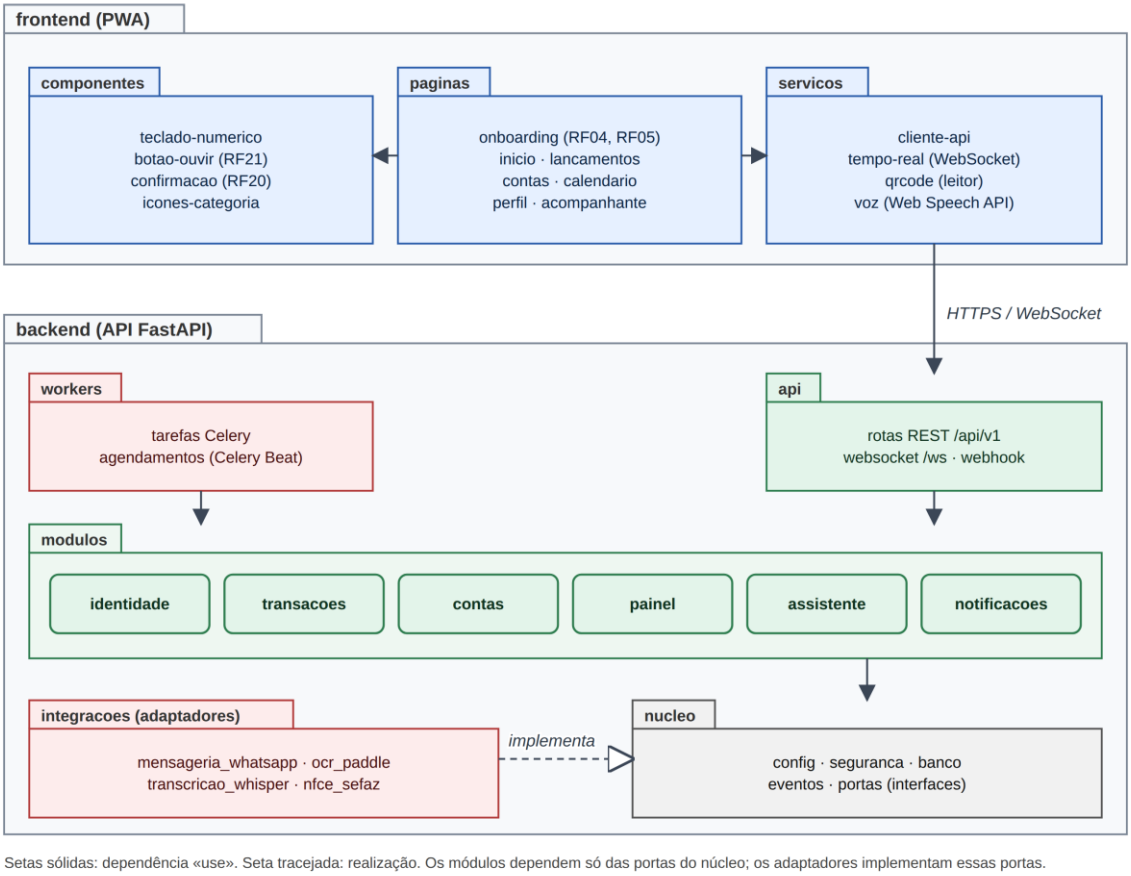
Módulo	Razão lógica	Requisitos	Comunica-se com
identidade	Concentra quem é o usuário e o que ele pode fazer; nenhum outro módulo decide sobre permissões.	RF01–RF03, RF06	Todos (fornece o usuário autenticado e seu perfil)
transacoes	Único dono das transações, para manter o saldo consistente nos dois canais.	RF07–RF11, RF17, RF18	painel, assistente, núcleo (eventos e porta NFC-e)
contas	Separa compromissos futuros (contas) de lançamentos já realizados (transações).	RF25, RF26, RF30, RF32–RF34	notificacoes, assistente, transacoes
painel	Cálculo de leitura reutilizado pelo app e pelo assistente.	RF19, RF22, RF23	transacoes, contas
assistente	Isola a conversa no WhatsApp; é o único módulo que conhece intenções e estados de diálogo.	RF12–RF16, RF23, RF27, RF31	transacoes, contas, painel, portas de mensageria, OCR e transcrição
notificacoes	Concentra regras de quando e como avisar, evitando avisos duplicados.	RF24, RF28, RF29	contas, painel, porta de mensageria
nucleo	Configuração, segurança, acesso ao banco, barramento de eventos e definição das portas.	RNF05	Usado por todos os módulos
integracoes	Adaptadores que implementam as portas: WhatsApp, PaddleOCR, faster-whisper e SEFAZ.	RF07, RF12–RF14, RF27	Implementam as portas do nucleo
frontend	Páginas, componentes acessíveis e serviços do PWA.	RF04, RF05, RF20, RF21 e telas	API (REST e WebSocket)

Fonte: elaborado pelo grupo (2026).

As formas de comunicação entre os módulos seguem quatro regras:

- O frontend fala com o back-end **somente** pela API REST e pelo WebSocket.
- Um módulo chama outro **apenas pela sua interface de serviço pública**; nunca lê ou grava tabelas de outro módulo.
- Mudanças que interessam a vários módulos (por exemplo, "transação criada") são publicadas como **eventos** no barramento.
- Qualquer acesso a serviço externo passa por uma **porta** do núcleo, implementada por um adaptador do pacote integracoes.

Figura 4 – Diagrama de pacotes



Setas sólidas: dependência «use». Seta tracejada: realização. Os módulos dependem só das portas do núcleo; os adaptadores implementam essas portas.

Fonte: elaborado pelo grupo (2026).

O Quadro 6 resume as principais rotas da API, que formam a interface entre a camada de apresentação e a de aplicação.

Quadro 6 – Principais rotas da API

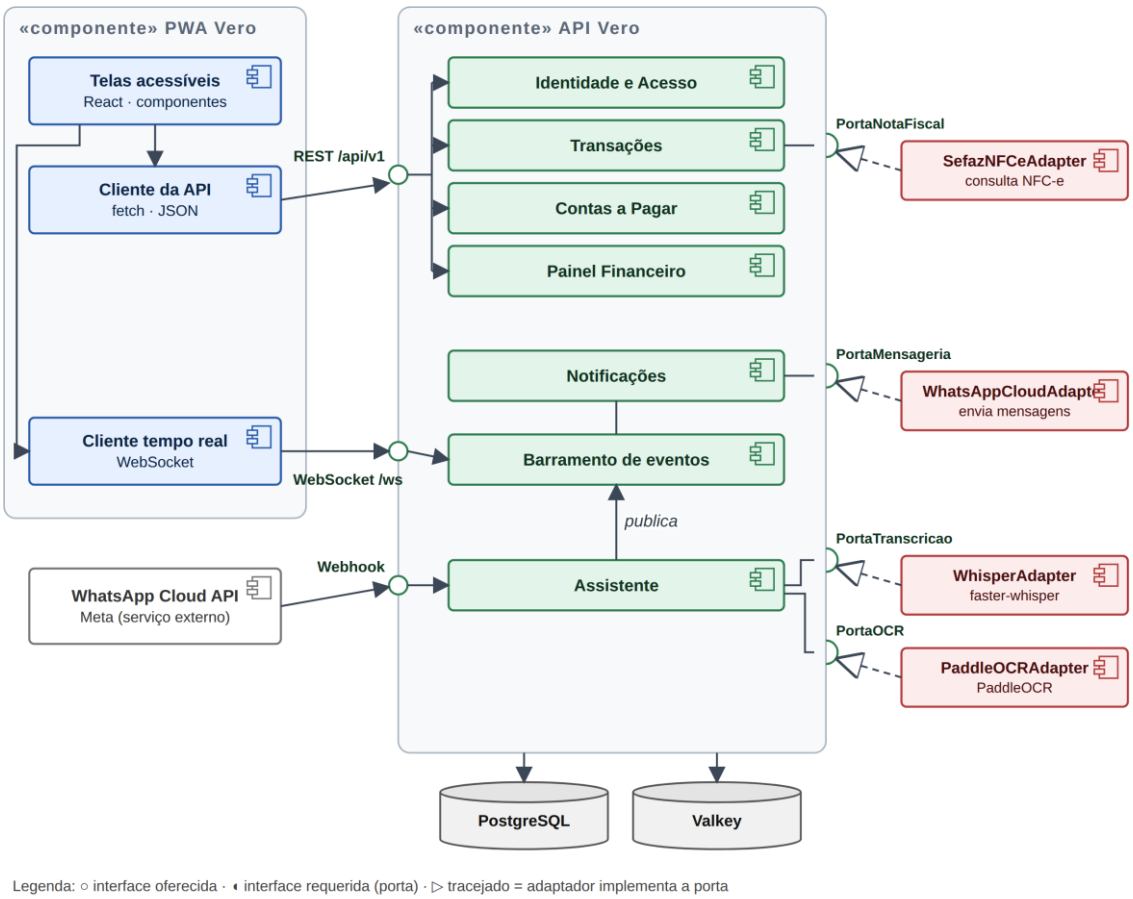
Rota	Finalidade	Requisitos
POST /api/v1/auth/cadastro e /auth/verificar	Cadastro pelo celular e verificação por código	RF03
POST /api/v1/auth/pin e /auth/webauthn/*	Entrada com PIN ou biometria	RF01, RF02
GET /api/v1/painel/mes-atual	Dinheiro disponível e percentual gasto	RF19, RF22
GET, POST, PATCH, DELETE /api/v1/transacoes	Histórico, registro, edição, exclusão e desfazer	RF08–RF11
POST /api/v1/transacoes/nfce	Registro a partir do QR Code da nota fiscal	RF07
GET, POST /api/v1/contas e POST /contas/{id}/pagar	Contas a pagar, filtros, calendário e baixa	RF25, RF26, RF30, RF32–RF34
POST /api/v1/acompanhantes e DELETE /acompanhantes/{id}	Autorizar e revogar acompanhante	RF06
WS /ws	Eventos em tempo real para o app	RF17
POST /webhook/whatsapp	Entrada das mensagens do assistente	RF12–RF16, RF23, RF27, RF31

Fonte: elaborado pelo grupo (2026).

2.5.3 Visão estrutural

A Figura 5 apresenta o diagrama de componentes. O PWA oferece telas acessíveis e usa dois componentes de comunicação: o cliente da API (REST) e o cliente de tempo real (WebSocket). A API oferece três interfaces — REST, WebSocket e webhook — e requer quatro portas, implementadas pelos adaptadores de integração. Essa separação permite, por exemplo, executar todos os testes do assistente no CI com adaptadores falsos.

Figura 5 – Diagrama de componentes

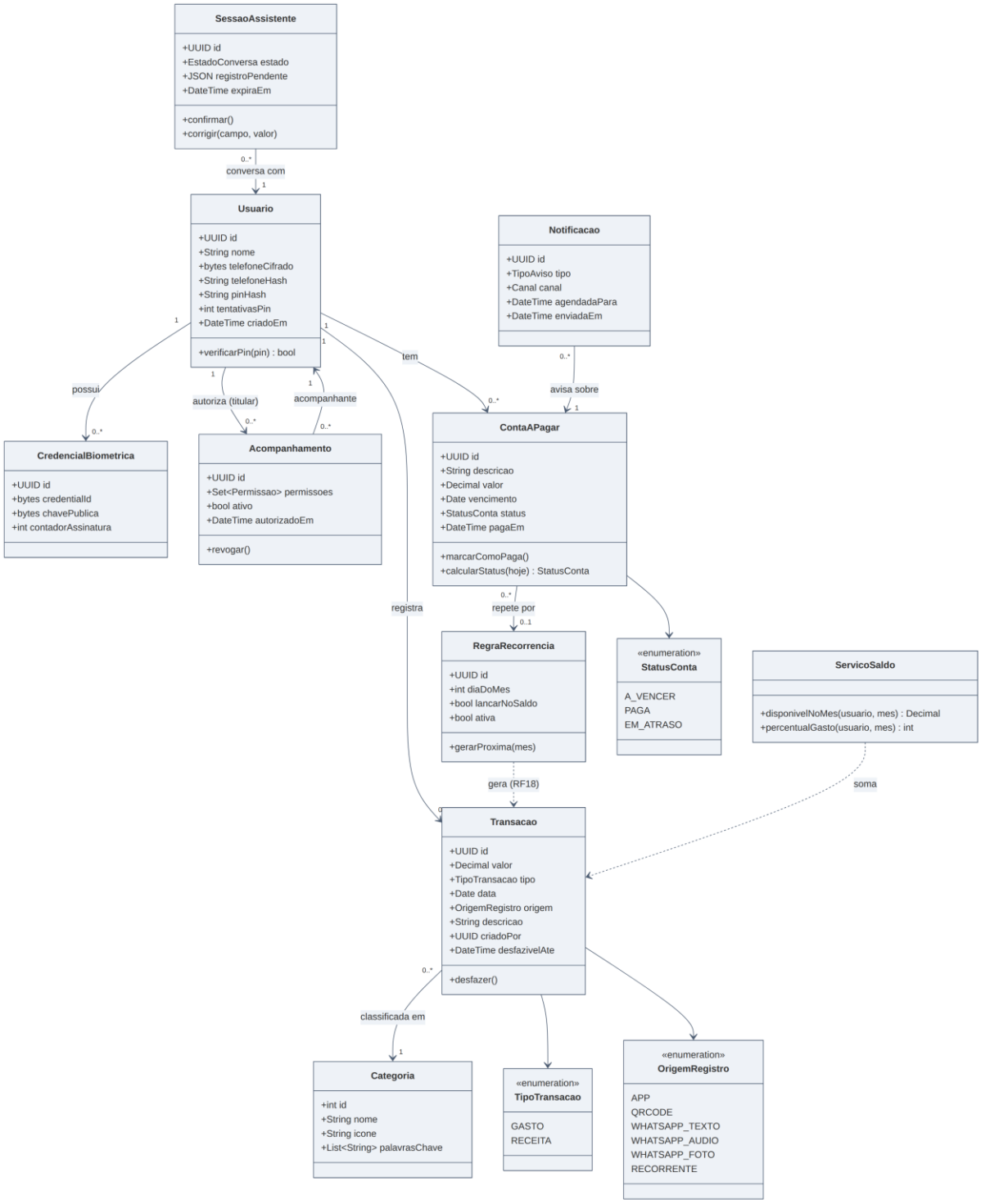


Fonte: elaborado pelo grupo (2026).

A Figura 6 apresenta o diagrama de classes do domínio. Destacam-se as seguintes decisões:

- **Usuario** guarda o telefone cifrado e, separadamente, um hash do telefone usado para localizar o usuário quando chega uma mensagem do WhatsApp, sem expor o número no banco.
- **Acompanhamento** liga o titular ao acompanhante com um conjunto explícito de permissões, que podem ser revogadas a qualquer momento (RF06).
- **Transacao** registra a origem (app, QR Code, texto, áudio, foto ou lançamento recorrente) e quem a criou, o que permite auditoria e o botão de desfazer por 10 segundos (RF11).
- **SessaoAssistente** guarda o estado da conversa e o registro pendente de confirmação, com prazo de expiração (RF15 e RF16).
- **ServicoSaldo** concentra o cálculo do disponível, usado tanto pela tela inicial quanto pelo assistente.

Figura 6 – Diagrama de classes do domínio



Fonte: elaborado pelo grupo (2026).

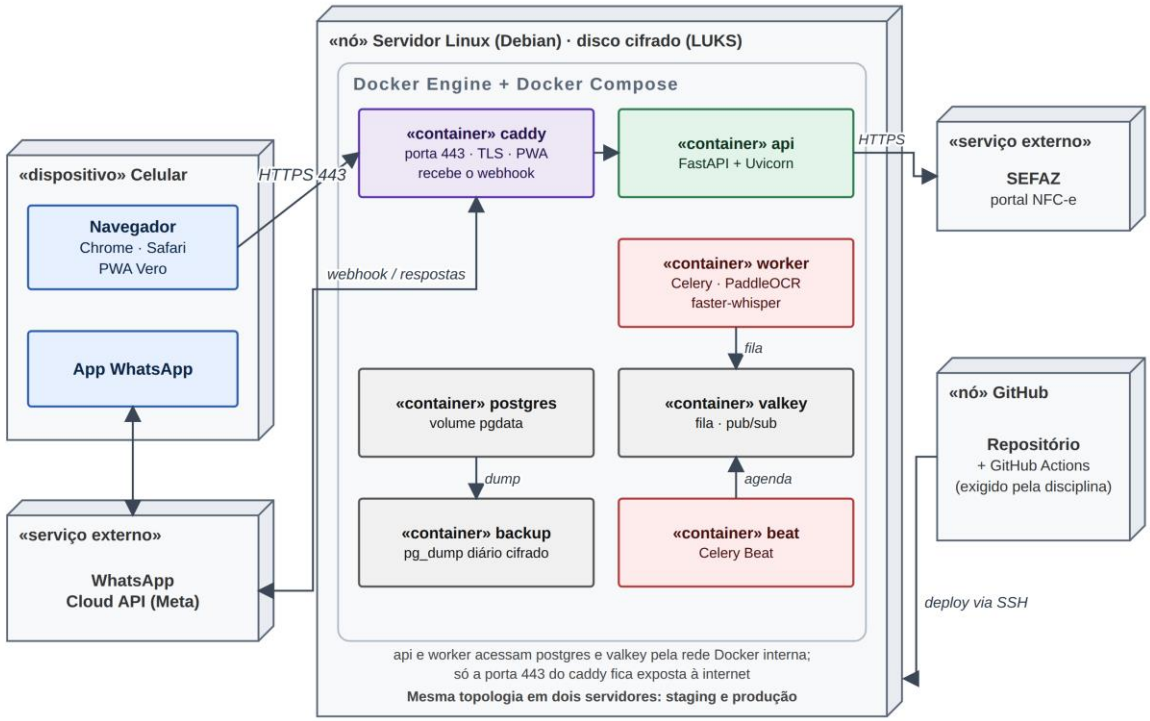
2.6 Visão de Implantação

O software será implantado em **servidores Linux (Debian)**, um para staging e outro para produção, com a mesma topologia. Cada servidor executa os serviços em contêineres orquestrados pelo **Docker Compose**, como mostra a Figura 7. A configuração estimada é de 4 vCPUs, 8 GB de RAM e 80 GB de disco SSD, dimensionada para manter os modelos de OCR e de transcrição em memória; esse valor será confirmado nas medições de desempenho da

Sprint 4. O provedor pode ser um servidor da universidade ou uma máquina virtual de qualquer provedor, pois toda a pilha é portátil.

O banco de dados é o **PostgreSQL**, escolhido por ser relacional e transacional (ACID), o que é essencial para valores financeiros, por oferecer tipos numéricos exatos e por ser maduro, bem documentado e de código aberto. O disco do servidor é cifrado com **LUKS**, e um contêiner de backup gera cópias diárias cifradas do banco.

Figura 7 – Diagrama de implantação



Fonte: elaborado pelo grupo (2026).

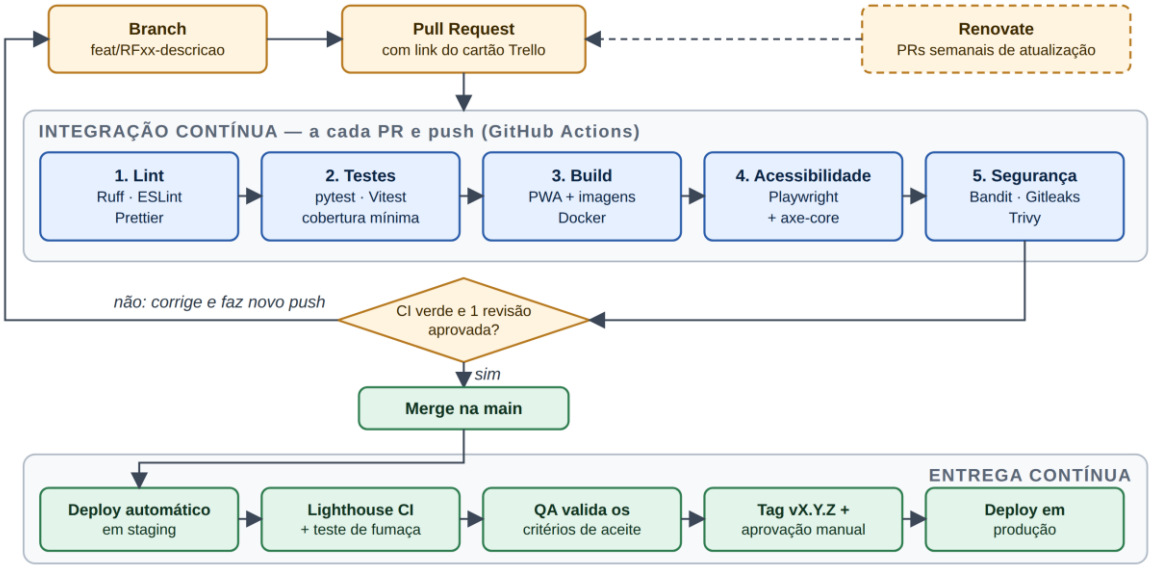
Quadro 7 – Ambientes

Ambiente	Onde executa	Como é atualizado	Dados e WhatsApp
Desenvolvimento	Máquina de cada integrante, com Docker Compose	Manualmente pelo desenvolvedor	Dados fictícios; adaptadores falsos ou número de teste
Staging	Servidor de staging	Automaticamente a cada merge na main	Dados fictícios; número de teste da Cloud API, usado pela QA
Produção	Servidor de produção	Por tag de versão (vX.Y.Z), com aprovação manual	Dados reais; número oficial do Vero

Fonte: elaborado pelo grupo (2026).

A esteira de integração e entrega contínuas (CI/CD), apresentada na Figura 8, garante que nenhuma alteração chegue à branch principal sem passar por lint, testes, build, verificação de acessibilidade e verificação de segurança, e que toda alteração aprovada chegue automaticamente ao ambiente de staging para validação da QA.

Figura 8 – Esteira de CI/CD



Fonte: elaborado pelo grupo (2026).

2.7 Restrições adicionais

Restrições negociais e de contexto. O Vero é acessado diretamente pela internet e exige identificação do usuário (PIN ou biometria). O sistema é aberto ao público em geral, mas o público prioritário — pessoas idosas e pessoas com pouca familiaridade com tecnologia — torna a usabilidade a característica de qualidade mais importante. O sistema é um produto acadêmico; a carga esperada é de dezenas de usuários simultâneos, e a arquitetura permite aumentar o número de workers caso a carga cresça.

Serviços externos não livres. A restrição de usar apenas software livre vale para todo o software do Vero. Três serviços externos ficam fora dela porque são impostos pelo problema ou pela disciplina:

- **WhatsApp:** o canal é exigido pelos requisitos, e a rede do WhatsApp é proprietária. A integração usa a **WhatsApp Cloud API**, a interface oficial da Meta, por meio de um adaptador próprio e aberto (WhatsAppCloudAdapter). A interface oficial foi escolhida por respeitar os termos de uso; gateways não oficiais podem ter o número bloqueado. Como a integração está isolada pela Porta de Mensageria, trocar de provedor afeta apenas o adaptador. Os modelos de mensagem para avisos e resumos precisam ser aprovados pela Meta antes da Sprint 4.
- **SEFAZ:** portal público de consulta da NFC-e, necessário para o RF07.
- **GitHub e GitHub Actions:** plataforma exigida pela disciplina. A esteira usa somente ferramentas livres e pode ser migrada para alternativas abertas, como Forgejo e Woodpecker CI, sem mudança na arquitetura.

Características de qualidade. O Quadro 8 resume as características de qualidade mais relevantes e como a arquitetura as sustenta.

Quadro 8 – Características de qualidade

Característica	Por que é relevante	Como a arquitetura atende
Usabilidade	Público geral, com foco em pessoas idosas e com baixo letramento.	PWA acessível, linguagem simples, confirmação visual (RF20), leitura em voz (RF21), tour em áudio (RF05).
Confiabilidade	Erros de registro afetam o dinheiro do usuário.	Confirmação antes de salvar (RF15), desfazer por 10 s (RF11), tarefas idempotentes, backup diário e transações no banco.
Desempenho	Resposta lenta faz o usuário desistir do assistente.	Workers assíncronos, modelos em memória e meta de 5 s medida desde a Sprint 4.
Manutenibilidade	Equipe de estudantes com sprints semanais.	Monólito modular com fronteiras claras, testes automatizados e padrões de código no CI.
Portabilidade	Usuários com celulares variados; hospedagem ainda indefinida.	PWA em qualquer navegador moderno; toda a pilha em contêineres Docker.
Segurança	Dados financeiros e pessoais (LGPD).	Ver a seguir.

Fonte: elaborado pelo grupo (2026).

Segurança e perfis de acesso. O Vero trata dados pessoais e financeiros e, por isso, segue a Lei Geral de Proteção de Dados (BRASIL, 2018). As medidas adotadas são:

- **Criptografia em trânsito:** HTTPS em todas as conexões externas, com certificados automáticos do Caddy; a rede interna entre contêineres não é exposta.
- **Criptografia em repouso:** disco do servidor cifrado com LUKS, telefone cifrado no nível da aplicação e backups cifrados.
- **Credenciais:** PIN guardado com hash Argon2 e bloqueio após tentativas erradas; a biometria usa WebAuthn, e o servidor guarda apenas a chave pública, nunca a digital ou o rosto (W3C, 2021).
- **Minimização de dados:** fotos e áudios enviados ao assistente são apagados após o processamento; os logs não registram telefone nem valores.
- **Direitos do titular:** consentimento explicado em linguagem simples no cadastro e opção de excluir a conta e todos os dados.
- **Segredos:** tokens da Cloud API, chaves de criptografia e senhas do banco ficam somente em variáveis de ambiente e nos segredos do GitHub, nunca no repositório; o Gitleaks bloqueia commits com segredos.

Quadro 9 – Perfis de acesso e permissões

Perfil	Pode	Não pode
Usuário (titular)	Tudo sobre os próprios dados; autorizar e revogar acompanhantes; excluir a própria conta.	Acessar dados de outros usuários.
Acompanhante	Ver saldo e contas do titular que o autorizou; registrar e corrigir lançamentos (RF06). Cada ação fica registrada com sua autoria.	Autorizar outros acompanhantes, alterar PIN ou biometria do titular, excluir a conta ou ver dados de outros titulares.
Assistente (sistema)	Agir em nome do usuário identificado pelo telefone, apenas após confirmação (RF15).	Gravar sem confirmação ou responder a números não cadastrados com dados financeiros.

Perfil	Pode	Não pode
Equipe técnica	Acessar o servidor por SSH com chave para manutenção e implantação.	Consultar dados de produção sem registro e sem necessidade de suporte.

Fonte: elaborado pelo grupo (2026).

3 Bibliografia

BASS, Len; CLEMENTS, Paul; KAZMAN, Rick. **Software architecture in practice**. 4. ed. Boston: Addison-Wesley, 2021.

BRASIL. Lei nº 13.709, de 14 de agosto de 2018. Lei Geral de Proteção de Dados Pessoais (LGPD). **Diário Oficial da União**, Brasília, DF, 15 ago. 2018. Disponível em: https://www.planalto.gov.br/ccivil_03/_ato2015-2018/2018/lei/l13709.htm. Acesso em: 4 out. 2026.

COCKBURN, Alistair. **Hexagonal architecture**. 2005. Disponível em: https://alistair.cockburn.us/hexagonal-architecture/. Acesso em: 4 out. 2026.

FOWLER, Martin. **MonolithFirst**. 2015. Disponível em: https://martinfowler.com/bliki/MonolithFirst.html. Acesso em: 4 out. 2026.

GRUPO ALLEN. **Vero**: documento de visão do produto e do projeto; requisitos de software v2; backlog v2; planejamento v2. Brasília: Universidade de Brasília, 2026. Documentos internos da disciplina MDS.

KRUCHTEN, Philippe. The 4+1 view model of architecture. **IEEE Software**, v. 12, n. 6, p. 42-50, nov. 1995.

NOTTINGHAM, Mark; WILDE, Erik; DALAL, Sanjay. **RFC 9457**: Problem Details for HTTP APIs. [S. l.]: IETF, 2023. Disponível em: https://www.rfc-editor.org/rfc/rfc9457. Acesso em: 4 out. 2026.

OPEN SOURCE INITIATIVE. **The Open Source Definition**. Disponível em: https://opensource.org/osd. Acesso em: 4 out. 2026.

PRESSMAN, Roger S.; MAXIM, Bruce R. **Engenharia de software**: uma abordagem profissional. 9. ed. Porto Alegre: AMGH, 2021.

SOMMERVILLE, Ian. **Engenharia de software**. 10. ed. São Paulo: Pearson Education do Brasil, 2018.

W3C. **Web Content Accessibility Guidelines (WCAG) 2.1**. 2018. Disponível em: https://www.w3.org/TR/WCAG21/. Acesso em: 4 out. 2026.

W3C. **Web Authentication**: an API for accessing public key credentials – Level 2. 2021. Disponível em: https://www.w3.org/TR/webauthn-2/. Acesso em: 4 out. 2026.

Apêndice A – Padrões de codificação e de repositório

Os padrões abaixo valem para todo o código do Vero e são verificados automaticamente pela esteira de CI sempre que possível.

Quadro 10 – Padrões de codificação e de repositório

Tema	Padrão adotado
Python (back-end)	PEP 8, aplicado pelo Ruff (lint e formatação). Anotações de tipo obrigatórias em funções públicas. Nomes do domínio em português, sem acentos (transacao, conta_a_pagar).
TypeScript (front-end)	ESLint + Prettier. Componentes funcionais em PascalCase; variáveis e funções em camelCase. Textos da interface somente em arquivos de mensagens. Todo elemento interativo com rótulo acessível.
API	Rotas no plural e em kebab-case; JSON em snake_case; valores em centavos inteiros; datas em ISO 8601; erros no formato da RFC 9457; versão no caminho (/api/v1).
Banco de dados	Migrações versionadas com Alembic; uma migração já aplicada nunca é alterada; dados de exemplo apenas fictícios.
Branches	GitHub Flow: main sempre estável; branches feat/, fix/, docs/ e ci/ com o ID do requisito (ex.: feat/RF03-cadastro-celular).
Commits	Conventional Commits (feat:, fix:, docs:, test:, ci:, refactor:).
Pull requests	Template com link do cartão do Trello, requisitos atendidos e como testar; pelo menos 1 aprovação e CI verde; merge por squash.
Testes	Pirâmide de testes: unitários para regras de negócio, integração para API e banco, ponta a ponta para fluxos críticos. Nome no formato test_<comportamento_esperado>. Cobertura mínima de 70% no back-end.
Dependências	Somente licenças aprovadas pela OSI; atualizações propostas semanalmente pelo Renovate.

Fonte: elaborado pelo grupo (2026).